

Line Intersection and Least Squares Approximation

Technical Brief

January 29, 2026

1 Intersection of Two Lines

In a deterministic 2D setting, two lines L_1 and L_2 defined by the equations $a_i x + b_i y = c_i$ intersect at a unique point $\mathbf{x}$ if the system is non-singular.

In higher dimensions or parametric contexts, we represent lines as:

$$\mathbf{L}_i(t) = \mathbf{p}_i + t\mathbf{d}_i \quad (1)$$

where $\mathbf{p}_i$ is a point on the line and $\mathbf{d}_i$ is a unit direction vector ($\|\mathbf{d}_i\| = 1$). For two lines in 3D, a unique intersection exists only if the lines are coplanar and non-parallel. If they are skew, we often seek the point of closest approach.

2 Least Squares for Multiple Lines

In practical applications such as computer vision (triangulation) or robotics, we often have N lines that, due to noise, do not intersect at a single point. We seek a point $\mathbf{x}$ that minimizes the sum of squared distances to all lines.

2.1 Objective Function

The vector from a point $\mathbf{p}_i$ on the line to the target point $\mathbf{x}$ is $(\mathbf{x} - \mathbf{p}_i)$. The projection of this vector onto the line's direction $\mathbf{d}_i$ is:

$$\text{proj}_{\mathbf{d}_i}(\mathbf{x} - \mathbf{p}_i) = ((\mathbf{x} - \mathbf{p}_i) \cdot \mathbf{d}_i)\mathbf{d}_i \quad (2)$$

The squared perpendicular distance D_i^2 from $\mathbf{x}$ to the i -th line is the magnitude of the rejected component:

$$D_i^2 = \|(\mathbf{x} - \mathbf{p}_i) - ((\mathbf{x} - \mathbf{p}_i) \cdot \mathbf{d}_i)\mathbf{d}_i\|^2 \quad (3)$$

Using the identity $(\mathbf{x} \cdot \mathbf{d})\mathbf{d} = (\mathbf{d}\mathbf{d}^T)\mathbf{x}$, we define the projection matrix $P_i = \mathbf{d}_i\mathbf{d}_i^T$. The distance squared can be rewritten as:

$$D_i^2 = \|(I - P_i)(\mathbf{x} - \mathbf{p}_i)\|^2 \quad (4)$$

2.2 Normal Equations

To find the global minimum, we minimize $S(\mathbf{x}) = \sum_{i=1}^N D_i^2$. Since $(I - P_i)$ is symmetric and idempotent ($(I - P_i)^2 = I - P_i$), the gradient ∇S with respect to $\mathbf{x}$ is:

$$\nabla S = 2 \sum_{i=1}^N (I - P_i)(\mathbf{x} - \mathbf{p}_i) = 0 \quad (5)$$

Rearranging into the form $A\mathbf{x} = \mathbf{b}$ yields:

$$\left(\sum_{i=1}^N (I - \mathbf{d}_i\mathbf{d}_i^T) \right) \mathbf{x} = \sum_{i=1}^N (I - \mathbf{d}_i\mathbf{d}_i^T) \mathbf{p}_i \quad (6)$$

This is a 3×3 (or 2×2) linear system that can be solved efficiently using Cholesky (LLT) or LDLT decomposition.

3 Modern C++ Implementation

The following implementation utilizes `std::transform_reduce` with `std::execution::par_unseq` for parallelized construction of the system matrices, reflecting modern C++20 paradigms.

```
#include <vector>
#include <numeric>
#include <execution>
#include <Eigen/Dense>

struct Line {
    Eigen::Vector3d p; // Point on line
    Eigen::Vector3d d; // Unit direction vector
};

Eigen::Vector3d findCommonIntersection(const std::vector<Line>& lines) {
    struct Accumulator {
        Eigen::Matrix3d m_sum = Eigen::Matrix3d::Zero();
        Eigen::Vector3d v_sum = Eigen::Vector3d::Zero();
    };

    auto result = std::transform_reduce(
        std::execution::par_unseq,
        lines.begin(), lines.end(),
        Accumulator{},
        [](Accumulator a, Accumulator b) {
            return Accumulator{a.m_sum + b.m_sum, a.v_sum + b.v_sum};
        },
        [](const Line& l) {
            Eigen::Matrix3d I_minus_ddT = Eigen::Matrix3d::Identity() - l.d * l.d.
                transpose();
            return Accumulator{I_minus_ddT, I_minus_ddT * l.p};
        }
    );

    return result.m_sum.ldlt().solve(result.v_sum);
}
```